

Heurística de Escalonamento (Greedy + Regras Locais)

1. Introdução

Este documento descreve o novo algoritmo apresentado na última reunião. O algoritmo adota uma abordagem *greedy* guiada por regras locais, composta por três fases:

- **Passo 1:** Satisfação dos requisitos mínimos (*min_required*).
- **Passo 2:** Aproximação dos requisitos ideais e preenchimento até ao alvo de 223 dias.
- **Passo 3:** Preenchimento de défices no número de dia de cada empregado

2. Conjuntos e Índices

Símbolo	Descrição
F	Conjunto de funcionários ($f = 1, \dots, n_F$)
D	Conjunto de dias ($d = 1, \dots, 365$)
S	Conjunto de turnos ($s = 1, \dots, n_S$)
E	Conjunto de equipas ($e = 1, \dots, n_E$)
H	Conjunto de dias especiais (domingos e feriados)

3. Parâmetros

Parâmetro	Descrição
$min_{d,s,e}$	Requisito mínimo
$ideal_{d,s,e}$	Requisito ideal
$vac_{f,d}$	1 se o funcionário f está de férias no dia d ; 0 caso contrário
A_f	Conjunto de equipas permitidas ao funcionário f
T	Total alvo de dias trabalhados (223)
C	Limite de dias especiais (22)

4. Variáveis

Variável	Descrição
$work_{f,d}$	1 se f trabalha no dia d ; 0 caso contrário
$shift_{f,d}$	Turno atribuído a f no dia d (0 se não trabalha)
$team_{f,d}$	Equipa atribuída a f no dia d (nula se não trabalha)
tw_f	Total de dias trabalhados por f
ts_f	Total de dias especiais trabalhados por f
$cov_{d,s,e}$	Número de funcionários já atribuídos a (d, s, e)

5. Regras de Viabilidade

A atribuição (f, d, s, e) é considerada viável se satisfizer simultaneamente:

- (1) Férias: $vac_{f,d} = 0$
- (2) Um turno por dia: $work_{f,d} = 0$
- (3) Equipa permitida: $e \in A_f$
- (4) Transição não tardia: $\begin{cases} shift_{f,d-1} \leq s & \text{se } work_{f,d-1} = 1, \\ s \leq shift_{f,d+1} & \text{se } work_{f,d+1} = 1, \end{cases}$
- (5) Máximo 5 dias consecutivos: $\sum_{k=d-r}^{d+r} work_{f,k} \leq 5$ para todo intervalo contíguo
- (6) Total $\leq T$: $tw_f + 1 \leq T$
- (7) Especiais $\leq C$: $ts_f + \mathbf{1}_{d \in H} \leq C$

6. Função de Pontuação (Heurística)

A função de pontuação é utilizada para escolher, entre todos os funcionários viáveis, aquele que deve ser atribuído ao par (d, s, e) . Define-se:

$$score(f, d) = tw_f + 3ts_f - 0.1(T - tw_f)_+ + 0.01 \cdot \text{rand}().$$

A heurística seleciona o funcionário com **menor** pontuação. Cada termo tem um papel específico:

6.1. Termo de carga total de trabalho

$$tw_f$$

Penaliza funcionários com muitos dias já atribuídos, promovendo um **equilíbrio global** na distribuição do trabalho.

6.2. Termo de dias especiais

$$3ts_f$$

Penaliza dias especiais (domingos e feriados) com peso três vezes superior, garantindo uma **distribuição justa** destes dias entre os funcionários.

6.3. Termo de subcarga

$$-0.1(T - tw_f)_+, \quad (x)_+ = \max(x, 0)$$

Favorece funcionários que estão abaixo do objetivo anual T (no caso do problema normal é $T = 223$). Quanto maior o défice $T - tw_f$, mais a pontuação diminui, incentivando a compensação de trabalhadores subutilizados. Quando $tw_f \geq T$, este termo passa a ser 0 e deixa de ter impacto.

6.4. Termo random

$$0.01 \cdot \text{rand}()$$

Pequena perturbação estocástica usada apenas para evitar empates e garantir **diversidade na solução**. O seu impacto na decisão é mínimo, servindo apenas como fator de desempate. Como tínhamos falado na reunião isto pode ser removido visto que a empresa prefere um algoritmo que produza sempre a mesma solução (ou manter mas usar uma seed).

6.5. Interpretação final

A combinação dos termos faz com que a heurística prefira funcionários que:

- trabalharam menos dias no total,
- trabalharam menos dias especiais,
- estão mais longe do objetivo anual,
- e, em situações de empate, sejam escolhidos de forma não determinística.

Esta função atua, portanto, como um mecanismo de **balanceamento** que guia a heurística a distribuir o trabalho de forma justa e consistente ao longo do ano.

7. Estrutura da Heurística

Passo 1: Cumprir mínimos

Enquanto $cov_{d,s,e} < min_{d,s,e}$: atribuir o melhor f viável.

Passo 2: Atingir ideais e aproximar dos 223 dias

Para cada funcionário f com déficit $T - tw_f$:

1. Preferir dias/turnos onde $cov_{d,s,e} < ideal_{d,s,e}$.
2. Se ainda existir déficit, aceitar qualquer slot viável.

Passo 3: Preenchimento residual

Se ainda existir déficit, permitir sobrecobertura:

$$cov_{d,s,e} \leftarrow cov_{d,s,e} + 1$$

desde que a atribuição continue viável.

8. Saída

O algoritmo devolve uma estrutura de atribuições, indexada por funcionário:

$$assign_f = \{(d, s, e) : work_{f,d} = 1\}.$$

Ou seja, para cada funcionário f , $assign_f$ contém todos os triplos (d, s, e) correspondentes aos dias d em que f trabalha ($work_{f,d} = 1$), indicando explicitamente em que turno s e em que equipa e foi escalado.

9. Pseudocódigo da Heurística

Entrada:

Conjuntos F, D, S, E, H ; parâmetros $min_{d,s,e}$, $ideal_{d,s,e}$, $vac_{f,d}$, A_f , alvo T , limite especial C .

- 1: **procedure** VIOLATESCONSECUTIVEDAYS(f, d) $\triangleright$ Função auxiliar que verifica se atribuir trabalho a f no dia d cria mais de 5 dias consecutivos
- 2: $n = |D|$
- 3: $runL = 0, i = d - 1$
- 4: **while** $i \geq 1$ **and** $work_{f,i} = 1$ **do**
- 5: $runL = runL + 1, i = i - 1$
- 6: **end while**
- 7: $runR = 0, i = d + 1$
- 8: **while** $i \leq n$ **and** $work_{f,i} = 1$ **do**
- 9: $runR = runR + 1, i = i + 1$
- 10: **end while**
- 11: $fullRun = runL + 1 + runR$
- 12: **return** ($fullRun > 5$)
- 13: **end procedure**

- 14: **procedure** ISFEASIBLE(f, d, s, e) $\triangleright$ Função que verifica se (f, d, s, e) respeita todas as restrições (regras laborais)
- 15: **if** $vac_{f,d} = 1$ **then return false**
- 16: **end if**
- 17: **if** $work_{f,d} = 1$ **then return false**
- 18: **end if**
- 19: **if** $e \notin A_f$ **then return false**
- 20: **end if** $\triangleright$ Transição não tardia
- 21: **if** $d > 1$ **and** $work_{f,d-1} = 1$ **and** $shift_{f,d-1} > s$ **then return false**
- 22: **end if**
- 23: **if** $d < |D|$ **and** $work_{f,d+1} = 1$ **and** $shift_{f,d+1} < s$ **then return false**
- 24: **end if** $\triangleright$ Máximo 5 dias consecutivos
- 25: **if** VIOLATESCONSEC(f, d) **then return false** $\triangleright$ Limites anuais de dias totais e dias especiais
- 26: **end if**
- 27: **if** $tw_f + 1 > T$ **then return false**
- 28: **end if**

```

29:   if  $d \in H$  and  $ts_f + 1 > C$  then return false
30:   end if
31:   return true
32: end procedure

33: function SCORE( $f, d$ ) ▷ Função de Pontuação
34:   return  $tw_f + 3 \cdot ts_f - 0.1 \cdot \max(T - tw_f, 0) + 0.01 \cdot \text{rand}()$ 
35: end function

36: procedure SOLVEHEURISTIC ▷ Início do Algoritmo
37:   for all  $f \in F, d \in D$  do
38:      $work_{f,d} = 0, shift_{f,d} = 0, team_{f,d} = \text{nulo}$ 
39:   end for
40:   for all  $f \in F$  do
41:      $tw_f = 0, ts_f = 0$ 
42:   end for
43:   for all  $d, s, e$  do
44:      $cov_{d,s,e} = 0$ 
45:   end for

Passo 1: cumprir requisitos mínimos

46:   for all  $d \in D$  do ▷ Percorre todos os dias
47:     for all  $s \in S$  do ▷ Percorre todos os turnos desse dia
48:       for all  $e \in E$  com  $min_{d,s,e} > 0$  do ▷ Só equipas com requisito mínimo
positivo
49:         while  $cov_{d,s,e} < min_{d,s,e}$  do ▷ Enquanto a cobertura estiver abaixo do
mínimo
50:            $Cands = \emptyset$ 
51:           for all  $f \in F$  do ▷ Construir conjunto de funcionários viáveis
52:             if ISFEASIBLE( $f, d, s, e$ ) then
53:               adicionar  $f$  a  $Cands$ 
54:             end if
55:           end for
56:           if  $Cands = \emptyset$  then
57:             break ▷ Não é possível cumprir este mínimo para  $(d, s, e)$ 
58:           end if
59:            $f^* = \arg \min_{f \in Cands} \text{SCORE}(f, d)$  ▷ Escolher  $f$  com menor score
60:            $work_{f^*,d} = 1$ 
61:            $shift_{f^*,d} = s$ 
62:            $team_{f^*,d} = e$ 
63:            $tw_{f^*} = tw_{f^*} + 1$ 
64:           if  $d \in H$  then  $ts_{f^*} = ts_{f^*} + 1$ 
65:           end if
66:            $cov_{d,s,e} = cov_{d,s,e} + 1$ 
67:         end while
68:       end for
69:     end for
70:   end for

```

71: **end procedure**

Passo 2: aproximar dos ideais e do alvo T

```

1: for all  $f \in F$  do                                ▷ Percorre todos os funcionários
2:    $deficit = T - tw_f$                                 ▷ Dias que faltam para o alvo  $T$  deste funcionário
3:   if  $deficit \leq 0$  then continue
4:   end if                                            ▷ Se já atingiu o alvo, passa ao próximo
5:   for  $round = 1$  até 2 do                            ▷ Round 1: prioriza slots abaixo do ideal; Round 2:
qualquer slot viável
6:     if  $deficit \leq 0$  then break
7:     end if
8:     for all  $d \in D$  do                                ▷ Percorre os dias do ano
9:       if  $deficit \leq 0$  then break
10:      end if
11:      if  $work_{f,d} = 1$  or  $vac_{f,d} = 1$  then continue
12:      end if                                        ▷ Ignora dias já trabalhados ou de férias
13:       $Slots = \{(s, e) : s \in S, e \in A_f\}$         ▷ Turnos/Equipas compatíveis com  $f$ 
nesses dias
14:      if  $round = 1$  then
15:        filtrar  $Slots$  para apenas pares com
         $cov_{d,s,e} < ideal_{d,s,e}$  se  $ideal_{d,s,e}$  definido,
        ou  $cov_{d,s,e} < min_{d,s,e}$  caso contrário    ▷ Round 1 só aceita slots ainda abaixo do
ideal (ou mínimo)
16:      end if
17:      aleatorizar ordem de  $Slots$ 
18:      for all  $(s, e) \in Slots$  do
19:        if ISFEASIBLE( $f, d, s, e$ ) then ▷ Verifica se atribuir  $(d, s, e)$  a  $f$  é viável
20:           $work_{f,d} = 1$ 
21:           $shift_{f,d} = s$ 
22:           $team_{f,d} = e$ 
23:           $tw_f = tw_f + 1$ 
24:          if  $d \in H$  then  $ts_f = ts_f + 1$ 
25:          end if
26:           $cov_{d,s,e} = cov_{d,s,e} + 1$                 ▷ Atualiza a cobertura desse slot
27:           $deficit = deficit - 1$                     ▷ Reduz o défice de dias do funcionário
28:          break                                    ▷ Passa ao próximo dia para este funcionário
29:        end if
30:      end for
31:    end for
32:  end for
33: end for

```

Passo 3: preencher défices restantes (mesmo com sobrecobertura)

```

34: for all  $f \in F$  do                                ▷ Nova passagem por todos os funcionários
35:    $deficit = T - tw_f$                                 ▷ Recalcula o défice após o Passo 2
36:   if  $deficit \leq 0$  then continue
37:   end if
38:   for all  $d \in D$  do                                ▷ Varre novamente todos os dias
39:     if  $deficit \leq 0$  then break

```

```

40:     end if
41:     if  $work_{f,d} = 1$  or  $vac_{f,d} = 1$  then continue
42:     end if ▷ Ignora dias já trabalhados ou de férias
43:      $Slots = \{(s, e) : s \in S, e \in A_f\}$  ▷ Todos os turnos/equipas permitidos para  $f$ 
    nesse dia
44:     aleatorizar ordem de  $Slots$  ▷ Novamente, ordem aleatória dos slots
45:     for all  $(s, e) \in Slots$  do
46:         if ISFEASIBLE( $f, d, s, e$ ) then ▷ Só verifica viabilidade individual; ignora
            se há sobrecobertura
47:              $work_{f,d} = 1$ 
48:              $shift_{f,d} = s$ 
49:              $team_{f,d} = e$ 
50:              $tw_f = tw_f + 1$ 
51:             if  $d \in H$  then  $ts_f = ts_f + 1$ 
52:             end if
53:              $cov_{d,s,e} = cov_{d,s,e} + 1$  ▷ Pode aumentar a cobertura acima do ideal
54:              $deficit = deficit - 1$ 
55:             break ▷ Passa ao próximo dia para este funcionário
56:         end if
57:     end for
58: end for
59: end for

60: construir  $assign_f = \{(d, s, e) : work_{f,d} = 1\}$  para cada  $f$  ▷ Constrói a estrutura final
    de atribuições
61: exportar e formatar tabela final de horários ▷ Gera o ficheiro/tabela de saída

```

11. Principais diferenças em relação ao algoritmo GreedyRandomized

Nesta secção estão as diferenças entre a nova heurística e o algoritmo GreedyRandomized. As diferenças principais podem ser agrupadas em quatro eixos: estrutura global, critério de escolha, tratamento da cobertura.

11.1. Estrutura global do algoritmo

- Heurística em três passos:

- Opera em **passos bem definidos**: (i) cumprir mínimos, (ii) aproximar ideais e alvo anual, (iii) preencher défices residuais mesmo com sobrecobertura.
- Em cada passo percorre *sistematicamente* dias, turnos e equipas, usando sempre a mesma função de pontuação para escolher o funcionário.

- GreedyRandomized:

- Baseia-se num ciclo externo *randomizado*, em que se escolhe aleatoriamente um funcionário admissível p e, de seguida, dias/turnos aleatórios para tentar novas atribuições.

11.2. Critério de escolha de funcionário e slot

- **Heurística em três passos:**

- Para cada slot (d, s, e) é avaliada uma lista de funcionários viáveis f .
- A escolha é **determinística-greedy**: seleciona-se o funcionário com menor valor da função

$$score(f, d) = tw_f + 3ts_f - 0.1(T - tw_f)_+ + 0.01 \cdot rand().$$

- O termo aleatório é apenas um desempate; o critério dominante é sempre o equilíbrio de carga e de dias especiais.

- **GreedyRandomized:**

- A escolha do funcionário é **aleatória**, dentro de subconjuntos prioritários (primeiro quem só pode estar numa equipa, depois em duas, etc.).
- Para cada funcionário escolhido, são testados alguns dias e turnos aleatórios; para cada tripla (d, s, t) viável, calcula-se uma função de “urgência de slot” $f2(d, s, t)$ baseada apenas no nível de cobertura (abaixo do mínimo, entre mínimo e ideal, acima do ideal).

11.3. Tratamento dos requisitos mínimos e ideais

- **Heurística em três passos:**

- Tem um **Passo 1** dedicado a preencher sistematicamente todos os $min_{d,s,e}$, sempre que existam funcionários viáveis.
- No **Passo 2** dá prioridade a slots com $cov_{d,s,e} < ideal_{d,s,e}$ (ou $< min_{d,s,e}$ se o ideal não existir), garantindo uma aproximação explícita aos requisitos ideais antes de aceitar sobrecobertura.
- O **Passo 3** só entra em ação quando ainda há défices individuais, permitindo então $cov_{d,s,e}$ acima do ideal.

- **GreedyRandomized:**

- Não tem uma fase separada de “cumprir mínimos”; em vez disso, a função $f2(d, s, t)$ atribui valor 0 a slots abaixo do mínimo, 1 a slots entre mínimo e ideal, e valores maiores a slots já cobertos acima do ideal.
- O algoritmo tenta **localmente** melhorar slots com $f2$ mais baixo, mas não percorre de forma exaustiva todos os requisitos mínimos/ideais.
- A satisfação dos mínimos e ideais depende do comportamento estocástico do ciclo de propostas e do tempo disponível.